

AI Native SOP 可视化看板前端形态调研报告

基于 Office Map、Role-based Workspace 与 Artifact Panel 的界面模式研究

维度	内容
报告目标	验证 AI Native SOP 看板第一版应采用何种前端界面模式，并为 mentor 的方向判断提供依据。
核心假设	前端不应被理解成单页 Dashboard，而应是 Office Map + Workspace + Artifact Panel 的三层结构。
研究对象	虚拟办公室、AI Workspace、Artifact 展示、Workflow/Handoff 可视化、Agent Observability。
推荐方向	轻游戏化 Office Map + 专业 Workspace Preview + 差异化 Artifact Panel。
适用阶段	客户演示 Demo 与内部方向确认，后续可扩展为真实工作入口。

一句话结论

第一版应优先做“客户能看懂、mentor 能判断方向、销售能讲故事”的前端演示，而不是一次性建设完整 AI 工作流平台。

0. Executive Summary / 核心结论

本次调研围绕 Office Map、Role-based Workspace、Artifact Panel 三层前端形态展开。结论是：该产品不宜被定义为传统 Workflow Dashboard，而应被定义为一个面向客户演示的 AI Native Workspace Map。它通过办公室地图降低理解成本，通过岗位工作空间展示 AI 与人类的协作过程，通过标准化 Artifact 面板证明各岗位产物可审查、可交接、可沉淀。

- 顶层适合采用 Office Map，用轻游戏化办公室隐喻让客户快速理解“每个岗位都有人和 AI 在工作”。
- 中层适合采用 Role-based Workspace，用 PM、Dev、QA 的岗位工作台承载真实工作过程的可信表达。
- 底层适合采用差异化 Artifact Panel，分别展示 PRD、Code Change、Test Report 等标准化业务对象。
- 第一版建议采用混合形态，而不是纯游戏化界面、纯 Workflow 流程图或纯专业 Workspace。
- MVP 应先聚焦 PM -> Dev -> QA 的单链路演示，证明 Artifact-driven handoff 能被客户理解。

推荐方向

Office Map + Role-based Workspace + Artifact Panel。顶层讲清楚组织如何运转，下钻展示岗位如何工作，Artifact 面板给出可审查的业务证据。

1. 项目背景与问题定义

1.1 背景

在 AI 原生工作流中，AI 与人类共同完成产品、开发、测试等环节。但对客户来说，AI 的工作过程往往不可见：他们只看到最终文档、代码或报告，很难理解 AI 在各岗位中具体创造了什么价值，也难以判断这些产物是否可以进入企业流程。

因此，本项目需要解决的不是“如何画一条流程线”，而是“如何让客户看到组织中的每个岗位都在被 AI 增强，并且每个岗位都产出可交接的标准化业务对象”。

1.2 当前问题

问题	说明	前端需要回答的问题
AI 工作不可见	客户看不到 AI 如何参与产品、开发和测试工作。	如何展示 AI 正在处理、等待确认、生成产物或遇到阻塞？

Workflow 过于抽象	传统流程图只表达节点流转，难以表达人和 AI 在岗位内协作。	如何把岗位空间、角色、状态和任务放在同一个认知框架里？
产物可信度不足	只展示“AI 在工作”不够，B 端客户还需要看到标准化业务产物。	如何让 PRD、代码变更、测试报告看起来专业、可审查、可交接？

1.3 本项目要解决的问题

如何通过一个前端可视化界面，让客户快速理解：每个岗位都有自己的 AI Workspace；人类员工与 AI Agent 在 Workspace 中协同工作；不同岗位之间不直接传递聊天上下文，而是通过标准化 Artifact 进行交接；管理者可以下钻查看每个阶段的产物、状态和证据。

2. 核心产品假设

2.1 我们设计的不是 Workflow，而是 Workspace

传统 Workflow 强调整节点之间如何流转，本项目更关注每个岗位内部的人机协作空间。一个岗位不是流程图上的一个圆点，而是一个可被进入、可被观察、可输出产物的 Workspace。

核心模型

Workspace（人 + AI Agent） -> Artifact -> Workspace（人 + AI Agent） -> Artifact -> Workspace（人 + AI Agent）

2.2 Workspace 不直接传递 Context，只输出 Artifact

每个 Workspace 内部可以包含对话、Prompt、工具调用、代码编辑、测试执行等复杂上下文，但这些过程不应直接传递给下游。下游只消费标准化 Artifact，例如 PRD、Code Change、Test Report。这样既减少跨岗位理解成本，也让企业可以审查、归档和复用每个阶段的产物。

2.3 三层前端形态假设

层级	作用	用户感知
Layer 1: Office Map	通过办公室地图展示 PM Office、Dev Office、QA Lab 等岗位空间。	一眼看懂组织在运转，每个岗位都有“人 + AI”在协作。
Layer 2: Workspace	点击办公室后进入岗位工作空间，展示任务、工具、AI 助手和工作过程。	相信这个岗位的 AI Workspace 是真的能工作的。

Layer 3: Artifact Panel

展示各岗位标准化输出，并支持版本、状态、来源、下游消费关系。相信产物可审查、可交接、可进入企业流程。

3. 调研目标与范围

3.1 调研目标

本次调研的目标不是寻找完全相同的产品，而是寻找能够支撑三层前端形态的界面模式和产品案例。最终报告应帮助 mentor 判断：第一版前端应偏向游戏化办公室、专业工作台、Artifact 证据层，还是三者结合。

3.2 调研范围

调研类别	对应层级	调研目的	代表案例
虚拟办公室 / AI 员工实体化	Office Map	研究如何让岗位与 AI 具象化。	WorkAdventure、Gather、My Virtual Office、AgentOffice
AI 工作区 / Coding Workspace	Workspace	研究人和 AI 如何在同一工作台协作。	Cursor、OpenHands、Replit Agent、Dify
Workflow / Handoff 可视化	Office Map + Workspace	研究流程状态、输入输出和 Artifact 交接。	n8n、Langflow、GitHub Actions
Artifact / Report / Review 面板	Artifact Panel	研究标准化产物如何被展示和审查。	Confluence PRD、GitHub PR、Allure、Playwright AgentPrism、
Agent Observability	高级下钻	研究 AI 工作过程如何作为证据被查看。	ClawMetry、LangSmith、Arize Phoenix

4. Interface Pattern Benchmark / 界面模式调研

4.1 Pattern 1: Office Map / 办公室地图模式

Office Map 解决的是第一眼认知问题：客户不需要先理解复杂的 Agent 架构，而是通过办公室、工位、角色、状态和 Artifact 交接，快速建立“组织正在运转”的心智模型。

案例	类型	可借鉴点	限制
WorkAdventure	开源虚拟办公室	可自定义地图、头像、空间互动、自托管，适合借鉴空间化组织表达。	偏远程办公和活动，不直接表达 AI 工作产物。
Gather	商业虚拟办公室	强调常驻空间、自然碰面和团队存在感，适合作为“办公室隐喻”的商业化参考。	更偏沟通协作，不适合直接照搬为 SOP 看板。
My Virtual Office	AI Agent 2D 办公室	直接将不可见的 Agent 工作转成可观察的办公室状态，契合“AI 正在干活”的演示目标。	仍偏开发者/概念产品，B 端信息设计需要加强。
AgentOffice	AI Agent 像素办公室	适合参考角色移动、任务状态、Agent 团队感和实时动画。	纯像素风有可信度风险。

启发：本项目顶层可以采用轻游戏化办公室地图，但动画必须事件驱动，只服务于开始工作、正在处理、等待审批、Artifact 生成、Artifact 递交、阻塞或失败等关键业务事件。

4.2 Pattern 2: Role-based Workspace / 岗位工作空间模式

Workspace 解决的是可信执行问题：客户点击某个岗位后，需要看到这个岗位不是一个动画角色，而是一个具有专业工具、上下文和输出能力的工作台。

案例	类型	可借鉴点	限制
Cursor	AI Coding Workspace	Chat、代码上下文、Agent 计划、编辑和 Diff 形成完整开发工作台体验。	主要服务开发角色，PM/QA 需要重新设计。
OpenHands	开源软件开发 Agent 平台	强调 Agent 计划、写代码、应用变更，适合参考透明工程任务执行。	偏技术用户，客户演示需要降噪。
Replit Agent	AI App Builder	对话、生成、预览、部署一体化，适合参考“从意图到可运行产物”的工作台叙事。	更像应用生成平台，不是企业 SOP 看板。
Dify	Agentic Workflow 平台	画布、节点、RAG、Agent 和工具连接适合参考可视化构建和调试。	更偏 AI 应用平台，不强调岗位办公室。

启发：第一版不应把 Workspace 做成完整生产级工作台，而应做成可信的 Workspace Preview。PM 展示 Chat + Doc Editor，Dev 展示 PRD 输入 + Code/Diff/Preview，QA 展示测试计划 + 执行结果 + 报告生成。

4.3 Pattern 3: Artifact Panel / 标准化产物面板模式

Artifact Panel 是本项目的证据层。游戏化 Office Map 只能让客户理解“有人在工作”，但 B 端客户还需要看到真实、结构化、可审查、可流转的业务对象。

岗位	Artifact 类型	应展示的核心字段	参考案例
PM	PRD / User Stories / Acceptance Criteria	业务目标、用户场景、功能范围、优先级、验收标准、依赖和风险。	Atlassian PRD / Confluence Template
Dev	Code Change / Pull Request / Build	提交说明、Diff、Changed Files、Commit、Review、Checks、构建状态。	GitHub Pull Request / GitHub Actions
QA	Test Report / Bug List / Regression Result	测试范围、通过率、失败用例、截图/附件、缺陷列表、回归结论。	Allure Report / Playwright HTML Report

启发：Artifact 不能统一设计成普通文件卡片。它们可以共享元信息骨架，例如类型、来源 Workspace、状态、版本、下游消费者和证据，但内容区必须岗位差异化。

4.4 Pattern 4: Workflow / Handoff 可视化模式

Handoff 解决的是跨岗位交接问题。岗位之间不是共享聊天上下文，而是通过 Artifact 递交、接收、审核、退回和版本迭代完成协作。

案例	可借鉴点	对本项目的启发
n8n	节点、输入输出、执行状态、错误处理和可追踪画布。	Artifact 交接可以借鉴节点状态与路径表达，但不宜变成复杂流程编辑器。
Langflow	低代码组件、节点连接、AI 工作流原型搭建。	适合参考 PM -> Dev -> QA 的 Artifact Path 和下钻输入输出。
GitHub Actions	Workflow、Job、Step、Log、Status 构成成熟的执行证据体系。	QA / Build 类 Artifact 可以借鉴 Job/Step/Log 层级。

启发：递交动作应同时包含视觉动作、状态变化和 Artifact Inbox。比如 PM 生成 PRD 后，办公室地图中出现文件递交动画；Dev Office 状态变成 New PRD Received；Dev Workspace 内出现待接收 PRD 卡片。

4.5 Pattern 5: Agent Observability / AI 工作过程下钻

Agent Observability 解决的是过程可信问题。普通客户不需要在顶层看到完整 Trace，但管理者或技术用户可能需要进一步确认 AI 到底执行了哪些工具、花了多少时间、在哪一步失败。

案例	类型	可借鉴点	放置建议
AgentPrism	Agent Trace React 组件	把 LLM Call、Tool Execution、Plan、Action、Retry 从 JSON 转成可读时间线和图。	放在 Workspace 或 Artifact 的高级下钻中。
ClawMetry	Agent Observability Dashboard	展示 token 成本、子 Agent 活动、session 历史和运行状态。	用于管理者视角，不放在客户演示首屏。
LangSmith	LLM Observability	提供 traces、性能指标、成本、延迟和失败调试。	用于证明 AI 过程可追溯。
Arize Phoenix	AI Observability / Evaluation	通过 trace 展示模型调用、检索、工具使用和自定义逻辑。	作为证据层扩展。

启发：Agent Trace 是证据，不是主叙事。顶层 Office Map 讲“组织在运转”，Artifact Panel 讲“产物可交接”，Trace 只在需要证明细节时展开。

5. 重点案例分析

案例	对应层级	核心界面模式	可直接借鉴	需要规避
My Virtual Office	Office Map	将 AI Agent 放进 2D 办公室，展示活跃状态。	AI 员工常驻、办公室地图、Agent 状态。	不要停留在像素概念，需要企业级信息层级。
Gather / WorkAdventure	Office Map	通过地图、头像和空间关系表达团队存在感。	空间化认知、工位、房间和角色移动。	不应把沟通型虚拟办公室等同于 SOP 看板。
Cursor / OpenHands	Workspace	AI 与开发者在工作台中协作完成代码任务。	Chat + Context + Tool + Diff 的专业工作台结构。	不要让第一版陷入完整 IDE 研发。
GitHub PR	Artifact Panel	用 PR 集成 Commit、Diff、Review、Checks。	Dev Artifact 的结构化证据表达。	对非技术客户需要简化术语。
Allure / Playwright Report	Artifact Panel	以测试结果、失败详情、附件和趋势支撑质量判断。	QA Artifact 的可信报告形态。	避免测试细节过多挤压演示节奏。
n8n / Langflow	Handoff	用节点和连接展示输入输出、状态和流转。	Artifact 传递路径与状态可视化。	不要把客户演示做成复杂流程搭建器。
AgentPrism / LangSmith / Phoenix	Observability	用 Trace 展示 AI 执行链路。	高级下钻、过程证据、失败定位。	普通客户首屏不展示完整 Trace。

6. 对比矩阵

6.1 三层形态与案例映射

前端层级	目标	可借鉴案例	关键模式
Office Map	快速理解人机协作组织正在运转。	My Virtual Office、WorkAdventure、Gather、AgentOffice	空间化、角色化、状态气泡、事件驱动动效。
Workspace	承载岗位真实工作过程。	Cursor、OpenHands、Replit Agent、Dify	Chat + Editor/Canvas + Tools + Preview。
Artifact Panel	展示可信产物。	Confluence PRD、GitHub PR、Allure、Playwright	结构化文档、Diff、Report、版本、审核。
Handoff	表达上下游交接。	n8n、Langflow、GitHub Actions	节点状态、输入输出、Inbox、提交/接收/退回。
Observability	展示 AI 过程证据。	AgentPrism、ClawMetry、LangSmith、Phoenix	Trace、Tool Call、Cost、Latency、Retry。

6.2 不同前端方向对比

方向	优点	风险	适合场景	建议
纯 Workflow Dashboard	清晰、专业、易落地。	记忆点弱，客户不容易感知 AI 和人在协作。	内部管理。	不作为第一推荐。
纯游戏化办公室	生动、易懂、传播感强。	B 端可信度不足，容易像概念 Demo。	概念传播。	只借鉴隐喻和动效。
纯专业 Workspace	真实可用，能承载工作。	顶层认知不直观，演示门槛高。	员工实际工作入口。	作为中层能力表达。
Office Map + Workspace + Artifact Panel	兼顾理解、执行和可信度。	设计复杂度较高，需要控制 MVP 范围。	客户演示 + 内部管理 + 后续产品化。	推荐第一版采用。

7. 设计模式总结

- Workspace as Office：每个岗位被表达为办公室或工位，让组织结构具象化。
- Human + AI as Pair：每个工位不是只有人，也不是只有 Agent，而是人类员工 + AI 搭档。

- Artifact as Handoff Object：PRD、Code Change、Test Report 是岗位之间的交接对象，而不是普通文件。
- Status as Motion：用状态、动作、气泡和事件动效表达 AI 正在工作。
- Drill-down from Narrative to Evidence：顶层讲故事，下钻看证据。
- Role-based Permission：员工视角进入自己的 Workspace，管理者视角查看全局和 Artifact 链路，演示视角强化叙事路径。

8. 推荐产品方向

8.1 推荐方向

最终建议
第一版采用 Office Map + Role-based Workspace + Artifact Panel 的三层前端结构。它既能让客户快速理解 AI 原生 workflow，也能通过岗位差异化 Artifact 保证 B 端可信度。

8.2 推荐原因

原因	说明
客户理解成本低	Office Map 让客户快速理解每个岗位都在工作，每个岗位都有 AI 搭档。
B 端可信度高	Artifact Panel 展示 PRD、代码变更、测试报告等标准化产物。
演示记忆点强	比传统 Dashboard 更容易被客户记住，也更适合售前讲解。
可逐步落地	第一版只做 PM -> Dev -> QA 单链路，后续再扩更多岗位和真实数据。
保留产品化空间	后续可扩展 Employee View、Manager View、权限、真实 Agent Trace 和工作流接入。

8.3 第一版 MVP 范围建议

1. 首页展示 Product Office、Dev Office、QA Lab 三个办公室，以及 PRD -> Code Change -> Test Report 的 Artifact Handoff Path。
2. 每个办公室默认展示角色、当前状态、当前任务和最新 Artifact；复杂指标放在 hover 或右侧详情中。
3. 点击办公室进入 Workspace Preview，展示该岗位的典型工作界面，而不是完整生产工作台。

- 4. 点击 Artifact 打开差异化 Artifact Panel：PM 看 PRD，Dev 看 PR/Diff/Build，QA 看测试报告。
- 5. 递交动作同时表现为视觉动画、状态变化和下游 Artifact Inbox。

9. 风险与注意事项

风险	表现	应对策略
游戏化过度	办公室地图过于像小游戏，削弱 B 端可信度。	采用轻游戏化企业风，保留空间隐喻和事件动效，减少无意义热闹。
信息密度过高	首屏塞入太多日志、指标和任务，变成复杂 Dashboard。	顶层只展示角色、状态、当前任务、最新 Artifact，复杂信息下钻。
Artifact 同质化	所有产物都长成普通文件卡，无法体现岗位差异。	PM、Dev、QA 分别设计专业面板，同时共享元信息骨架。
Workspace 膨胀	第一版试图做完整 PM/Dev/QA 三个生产级工作台。	MVP 只做可信工作状态模拟和典型产物展示。
Trace 暴露过多	把技术执行细节放在客户首屏，增加理解负担。	Trace 只作为高级证据层，面向管理者或技术用户下钻。

10. 下一步计划

步骤	任务	产出
Step 1	完成案例池：围绕五类模式收集 20 个左右案例，并记录截图、链接、界面特征和可借鉴点。	候选案例池
Step 2	筛选 P0 案例：重点深挖 12-15 个最相关案例，形成案例卡片。	重点案例卡片
Step 3	整理模式库：沉淀 Office Map、Workspace、Artifact Panel、Handoff、Observability 的设计模式。	界面模式库
Step 4	输出方向建议：给出 2-3 个前端方向，并明确推荐第一版方案。	方向建议页
Step 5	制作概念原型：围绕 PM -> Dev -> QA 流程制作首页 Office Map、三个 Workspace Preview、三个 Artifact Panel 和 Handoff 动效示意。	概念原型

附录 A：建议的 Artifact 元信息骨架

元信息	说明
Artifact Type	PRD、User Story、Code Change、Build、Test Report 等。
Owner Workspace	产出该 Artifact 的岗位空间，例如 PM Office、Dev Office、QA Lab。
Status	Draft、Submitted、Accepted、Rejected、Blocked、Deprecated。
Version	v1、v2、v3，支持被退回和迭代。
Consumed By	下游消费方，例如 Dev Workspace 或 QA Workspace。
Evidence	生成过程摘要、人工审核记录、关键结论、Trace 链接。

附录 B：参考来源

- [1] **WorkAdventure GitHub**: <https://github.com/workadventure/workadventure>
- [2] **WorkAdventure Open Source**: <https://workadventure.re/open-source/>
- [3] **Gather Virtual Office**: <https://www.gather.town/virtual-office>
- [4] **My Virtual Office GitHub**: <https://github.com/eliautobot/my-virtual-office>
- [5] **My Virtual Office**: <https://myvirtualoffice.ai/>
- [6] **AgentOffice GitHub**: <https://github.com/harishkotra/agent-office>
- [7] **Cursor**: <https://cursor.com/>
- [8] **Cursor Product**: <https://cursor.com/product>
- [9] **OpenHands**: <https://www.openhands.dev/>
- [10] **OpenHands GitHub**: <https://github.com/OpenHands/openhands>
- [11] **Replit Agent**: <https://replit.com/products/agent>
- [12] **Dify**: <https://dify.ai/>
- [13] **Dify GitHub**: <https://github.com/langgenius/dify>
- [14] **Atlassian PRD Guide**: <https://www.atlassian.com/agile/product-management/requirements>
- [15] **Confluence PRD Template**: <https://www.atlassian.com/software/confluence/templates/product-requirements>
- [16] **GitHub Pull Request Docs**: <https://docs.github.com/articles/reviewing-proposed-changes-in-a-pull-request>
- [17] **GitHub Actions Docs**: <https://docs.github.com/actions>
- [18] **Allure Report**: <https://allurereport.org/>
- [19] **Playwright Test Reporters**: <https://playwright.dev/docs/test-reporters>
- [20] **n8n**: <https://n8n.io/>
- [21] **Langflow**: <https://www.langflow.org/>
- [22] **AgentPrism GitHub**: <https://github.com/evilmartians/agent-prism>
- [23] **ClawMetry**: <https://clawmetry.com/>
- [24] **LangSmith Observability**: <https://docs.langchain.com/langsmith/observability>
- [25] **Arize Phoenix**: <https://arize.com/docs/phoenix>